

บทที่ 7

การพัฒนาระบบไฟร์แทรป

7.1 แนวคิด

ไฟร์แทรป (FireTrap) เป็นระบบจำลองทำหน้าที่เป็นเครื่องจำลองที่จำลองตัวเองให้เหมือนเครื่องให้บริการจริงมากที่สุด โดยอาจมีการเปิดบริการหรือช่องโหว่ต่าง ๆ เพื่อให้ผู้บุกรุกหลงเชื่อและแสดงพฤติกรรมในการบุกรุกต่าง ๆ ซึ่งพฤติกรรมเหล่านั้นจะถูกบันทึกไว้และส่งกลับไปให้ยังส่วนเก็บข้อมูลและส่วนแสดงผลกลาง นอกจากนี้จะมีการตรวจสอบสภาพและความเสียหายของเครื่องกับดักอยู่ตลอดเวลา ถ้าถึงจุดที่เครื่องกับดักอ่อนแอ ผู้ดูแลสามารถที่จะระงับการใช้เครื่องกับดักนั้นโดยอัตโนมัติ

7.2 ขอบเขตความสามารถ

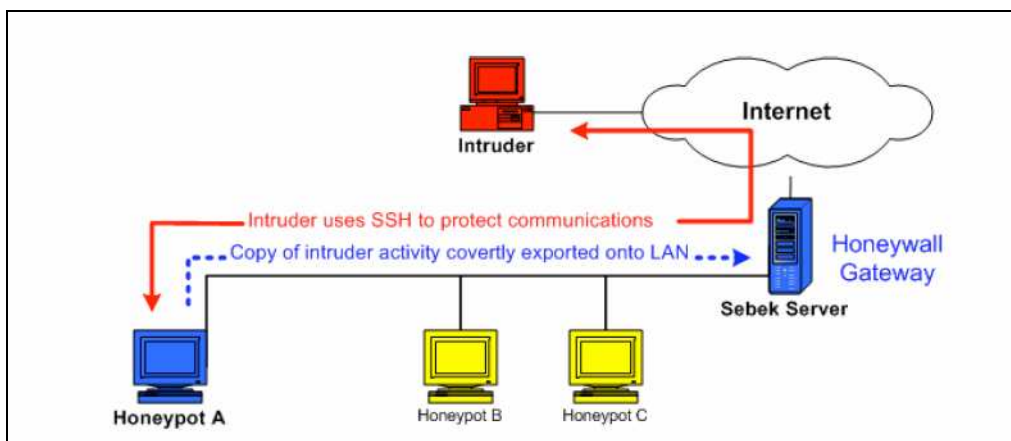
ไฟร์แทรป (FireTrap) เป็นระบบจำลองซึ่งแพ็คเกจที่ถูกระบบไฟร์สกรีนจำแนกว่าเข้าข่ายผู้บุกรุกจะถูกส่งมาที่ส่วนนี้ และทำการหลอกล่อให้ผู้บุกรุกเข้าใจผิดว่าเป็นเครื่องที่ให้บริการจริง โดยจะจำลองการบริการต่าง ๆ ให้เหมือนจริงมากที่สุด พร้อมทั้งเปิดบริการหรือช่องโหว่ เพื่อให้ผู้บุกรุกหลงเชื่อและแสดงพฤติกรรมในการบุกรุกต่าง ๆ ซึ่งพฤติกรรมเหล่านั้นจะถูกบันทึกจะส่งไปเก็บไว้ยังส่วนเครื่องแม่ข่ายฐานข้อมูล และจะมีการตรวจสอบสภาพและความเสียหายของเครื่องกับดักในไฟร์แทรปตลอดเวลาจากโปรแกรม S2I ซึ่งอยู่ในส่วนของไฟร์สกรีน

เครื่องกับดักในระบบไฟร์แทรปนี้ ทางผู้พัฒนาได้พัฒนาให้มีความปลอดภัยต่อองค์กร นั่นคือผู้บุกรุกจะไม่สามารถใช้เครื่องกับดักไปเป็นฐานโจมตีเครื่องอื่นๆ และพัฒนาให้มีความแนบเนียนเสมือนว่าเป็นเครื่องให้บริการจริงที่เป็นเป้าหมายของผู้บุกรุก โดยปกปิดความผิดปกติของค่าต่างๆ ที่ระบุว่าเป็นเครื่องกับดัก เช่น ไอพีแอดเดรส แมคแอดเดรส หรือไม่พบเซอร์วิสที่ทำให้ทราบว่าได้มีการเฝ้าดูการกระทำต่างๆ โดย ทุก ๆ keystroke ที่ผู้บุกรุกสั่งงานเข้ามา แม้ผู้บุกรุกจะใช้คำสั่งผ่านการเข้ารหัสก็ตาม หรือแม้แต่การนำเข้าไฟล์หรือส่งออก ตัวเครื่องไฟร์แทรปจะส่งข้อมูลทุกอย่างไปเก็บไว้ในเครื่องแม่ข่ายฐานข้อมูล ซึ่งการส่งข้อมูลออกไปจะส่งไปแบบเข้ารหัสและไร้ตัวตน โดยที่ผู้บุกรุกไม่ทราบว่าได้มีการส่งข้อมูลออกไป ด้วยเทคนิคของ Sebek ซึ่งทำงานในแบบ kernel-base นั่นคือแม้ว่าผู้บุกรุกจะนำโปรแกรมดักจับแพ็คเกจมาติดตั้งก็จะไม่พบว่าเครื่องไฟร์แทรปได้ส่งข้อมูลออกไป

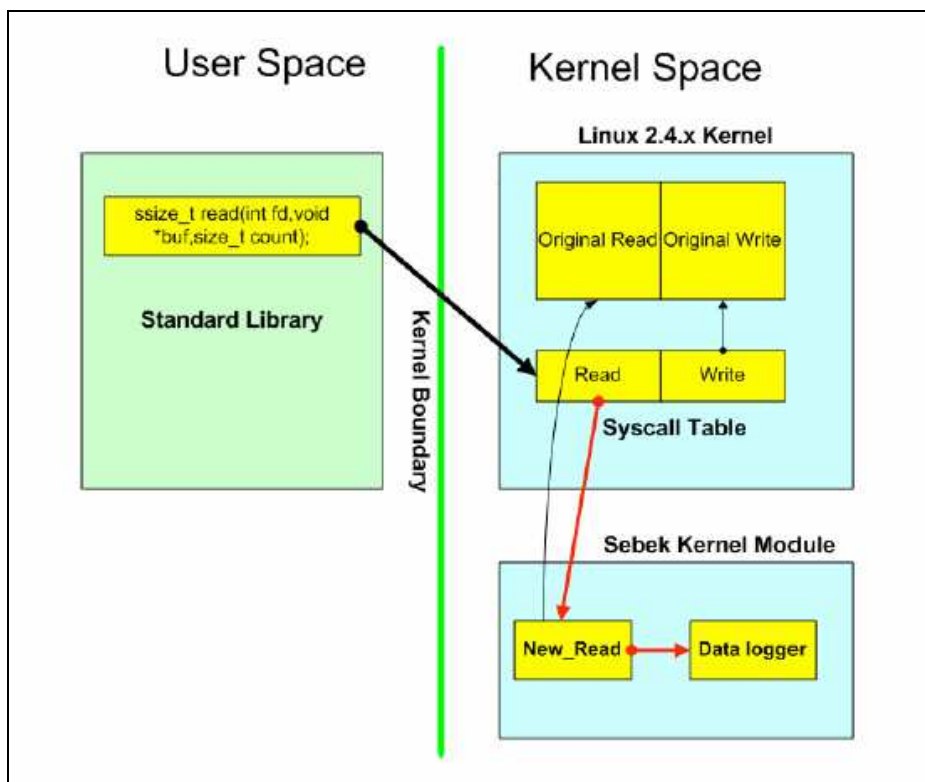
7.3 เทคโนโลยีที่ใช้ในการพัฒนาโปรแกรม

7.3.1 เซเบค (Sebek)

เซเบคโคไลเอนต์จะคอยซ่อนตัวอยู่ในกับดักเพื่อเฝ้าดูพฤติกรรมและการกระทำของผู้บุกรุก และคอยส่งข้อมูลนั้นไปยังเครื่องที่ใช้เก็บพฤติกรรมลงฐานข้อมูล โดยส่งผ่านทางที่มีความปลอดภัย และมีการเข้ารหัสข้อมูลในรูปแบบของเซเบคเอง

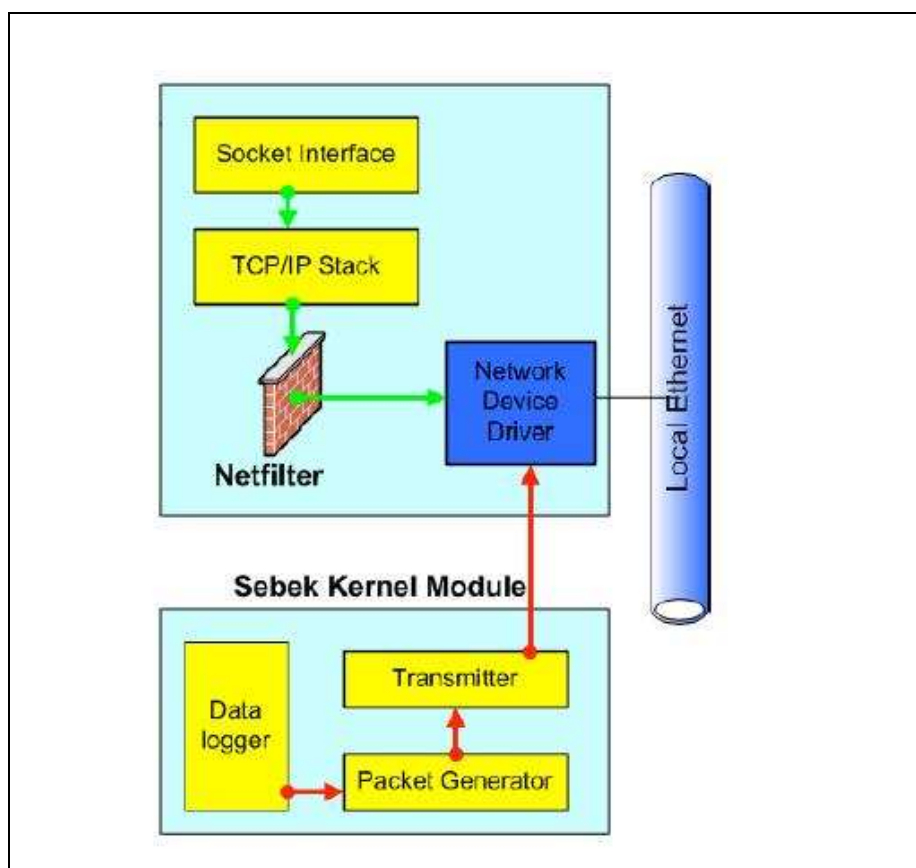


รูปที่ 7.1 แสดงการทำงานของ Sebek



รูปที่ 7.2 รูปภาพแสดงการcapture ข้อมูลที่ทำภายใน kernel

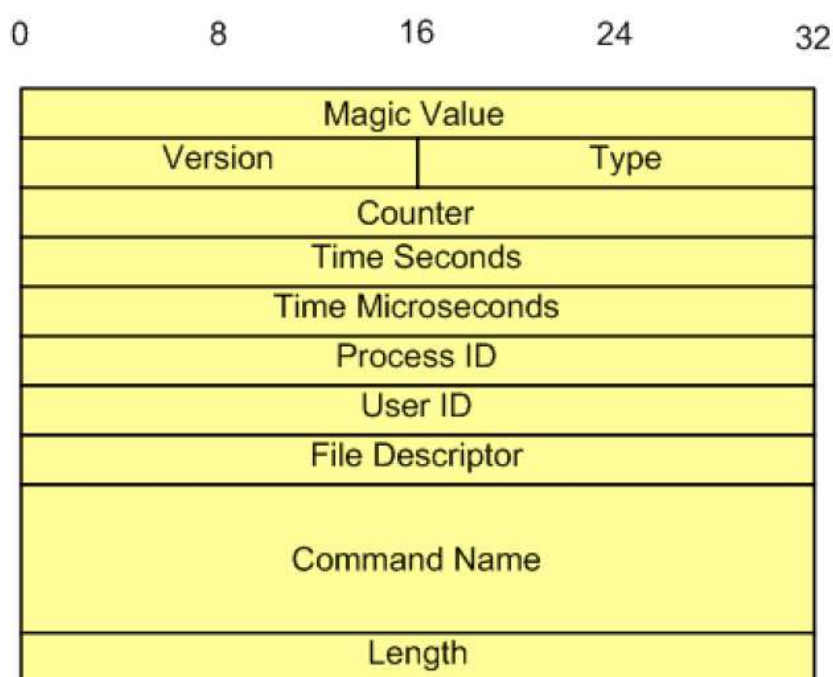
เซเบคเป็น Data Capture Tool แบบ Kernel-based โดยในการนำมาใช้จะประยุกต์ใช้เป็นแบบ LKM (Loaded Kernel Module) มีความโดดเด่นในความสามารถในการดักจับข้อมูลได้ทุกการกระทำของผู้บุกรุกที่กระทำผ่าน read() system call ไม่ว่าจะเป็นในส่วนของ keystroke, files transfer, burneye passwords หรือหากมีการใช้ IRC client หรือ e-mail client ตัวเซเบคก็สามารถดักจับได้ ดังเช่นเมื่อผู้บุกรุกมีการนำไฟล์ส่งเข้ามา ตัวเซเบคจะสามารถอ่านและบันทึกไฟล์นั้นไว้ได้ ทั้งยังสามารถใช้เป็น Glass-box เพื่อเปรียบเทียบกับการทำงานของ Black-box ที่เรารู้จัก นั่นคือเราสามารถติดตามการทำงานของโปรแกรมที่ผู้บุกรุกเอามารันได้ด้วย แม้ว่าผู้บุกรุกจะออกจากระบบไปแล้วก็ตาม นอกจากนี้ยังสามารถบันทึกพฤติกรรมของผู้บุกรุกที่กระทำผ่านทางช่องทางที่มีการเข้ารหัส เช่น SSH ได้ด้วย



รูปที่ 7.3 รูปภาพแสดงการby-passไม่ผ่าน TCP stack

เซเบคมีรูปแบบวิธีการปิดซ่อนตัวเองได้อย่างมีประสิทธิภาพ และค่อนข้างยากที่จะตรวจสอบพบทั้งจากภายในและภายนอก โดยจะทำการปิดซ่อนโมดูลที่ใช้ และลบ linked list ที่จะแสดงการมีตัวตนออก โดยใช้โมดูลที่ชื่อ the cleaner ทำให้ผู้บุกรุกไม่สามารถตรวจสอบพบว่เซเบคกำลังทำงานอยู่ภายในเครื่อง ตัวเซเบคจะมีฟังก์ชันในการสร้างและส่งแพ็คเกจเกิดของเซเบคเอง โดยไม่ใช้ TCP/IP stack ดังนั้นจึงไม่สามารถถูกมองเห็นและไม่สามารถทำการ block traffic ของ

เซเบคได้ หลังจากเซเบคสร้างแพ็คเก็ตเสร็จ แพ็คเก็ตจะถูกส่งไปยัง device driver โดยตรง ไม่ผ่าน raw socket interface ซึ่งทำให้พวกโปรแกรม sniffer ที่ใช้ libpcap เป็นฐานไม่สามารถเห็นได้ เพราะมันไปดักจับข้อมูลที่ raw socket interface และเพื่อป้องกันการตรวจจับในเครือข่าย เซเบคจะไม่ใช้ ARP แต่จะกำหนด MAC Address ของเซิร์ฟเวอร์สำหรับเก็บพฤติกรรมผู้บุกรุกไว้เลย เพื่อจะได้ไม่ต้องทำการ ARP request เพื่อป้องกันการทำ ARP spoofing ซึ่งจะลดการดักจับข้อมูลในเครือข่ายได้ ทั้งนี้เซเบคยังมีรูปแบบการส่งข้อมูลด้วยโปรโตคอลของตัวเองซึ่งใช้การส่งแบบโปรโตคอล UDP ในการสื่อสารระหว่างเซเบคไคลเอนต์และเซเบคเซิร์ฟเวอร์ โดยจะเป็นการสื่อสารทางเดียว คือ ส่งข้อมูลจากเซเบคไคลเอนต์ไปยังเซเบคเซิร์ฟเวอร์เท่านั้น โปรโตคอลของเซเบคมีรูปแบบดังต่อไปนี้



รูปที่ 7.4 รูปแสดงโปรโตคอลของเซเบค

ค่า Process ID , User ID และ File Descriptor จะเป็นค่าของโปรเซสที่ทำการส่งสัญญาณ read () system call นั้น ๆ จากเคอร์เนลมาให้ ฟิลด์ Length จะแปรเปลี่ยนไปตามความยาวข้อมูลที่ส่งมาโดยถ้าเป็นข้อมูลที่ส่งมาจากฟังก์ชัน read () จะยาวกว่าข้อมูลทั่วไป จากนั้นเซเบคจะตัดเอาข้อมูลเฉพาะข้อมูลที่เกี่ยวกับเรคคอร์ดของเซเบคมาประมวลผลเท่านั้น

ภายในตัวเซเบคไคลเอนต์จะมีการตัดสินใจว่าจะซ่อนแพ็คเก็ตเพื่อป้องกันการดักจับจาก user space หรือไม่ โดยพิจารณาจากว่าแพ็คเก็ตนั้นๆ ต้องเป็นแพ็คเก็ต UDP มีหมายเลขพอร์ตปลายทางและหมายเลขในฟิลด์ Magic value ที่ตรงกับค่าที่กำหนดไว้ในโปรแกรมเซเบค ซึ่งค่า Magic value นี้จะช่วยป้องกันการ brute force ในการดักจับแพ็คเก็ตได้

7.3.2 เซมเฮน (Samhain)

เซมเฮน เป็นระบบที่ใช้ในการตรวจสอบความถูกต้องของข้อมูลและใช้ในการแจ้งเตือนการบุกรุกระบบซึ่งสามารถใช้บนเครื่องโฮสต์เครื่องเดียว หรือใช้ในระบบเครือข่ายขนาดใหญ่ได้ ซึ่งในโครงการนี้ไม่ได้ใช้กับเครื่องโฮสต์เพียงเครื่องเดียว หากแต่จะใช้กับเครื่องหลายๆ เครื่องที่อยู่ในระบบเครือข่ายโดยจะมีเครื่องแม่ข่ายที่ทำหน้าที่เป็นศูนย์กลางคอยรับข้อมูลจากเครื่องลูกข่ายทั้งหลายในระบบเครือข่าย ซึ่งจะได้อธิบายรายละเอียดของโครงสร้างและการทำงานในส่วนถัดไป

7.3.2.1 การใช้งานในรูปแบบระบบเครือข่าย (network)

เซมเฮน เป็นระบบที่ถูกออกแบบมาให้ช่วยต่อการมอนิเตอร์โฮสต์หลายๆ เครื่องในเครือข่าย ซึ่งจะประกอบไปด้วยโปรเซสที่ทำงานในลักษณะที่เป็นเดมอนในโฮสต์แต่ละเครื่อง และจะมีเครื่องแม่ข่ายกลางที่จะคอยเก็บบันทึกหรือรายงานต่างๆ จากเดมอนเหล่านั้นผ่านทาง การเชื่อมต่อแบบที่ซีพี/ไอพี ซึ่งในการส่งข้อมูลจากเครื่องลูกข่ายต่างๆ ไปให้เครื่องแม่ข่ายกลางนั้น จะต้องมีการยืนยันตนก่อนเพื่อป้องกันการปลอมแปลงข้อมูลที่ส่งไปให้เครื่องแม่ข่าย โดยในขั้นแรกจะต้องตกลงกันในเรื่องของโปรโตคอลที่จะใช้ในการยืนยันตนก่อน จากนั้นจะมีการแลกเปลี่ยน session key กัน การเชื่อมต่อที่มาจากเครื่องโฮสต์ที่ไม่ได้ทำการลงทะเบียนไว้จะถูกครอบกั้นทันที และในอีกกรณีหนึ่งที่จะถูกครอบกั้นคือการเชื่อมต่อจากเครื่องโฮสต์ที่ได้มีการลงทะเบียนไปแล้ว แต่เครื่องลูกข่ายไม่สามารถทำการยืนยันตนได้สำเร็จ เมื่อ Session key ได้ถูกสร้างขึ้นมาเครื่องลูกข่ายจะใช้ session key นั้นในการเซ็นลงไปในข้อความของตนเพื่อใช้ในการยืนยันตนและเมื่อข้อความถูกส่งไปถึงเครื่องแม่ข่าย เครื่องแม่ข่ายจะทำการตรวจสอบลายเซ็นของเครื่องลูกข่าย จากนั้นเมื่อตรวจสอบพบว่าถูกต้องแล้วเครื่องแม่ข่ายจะเอาลายเซ็นนั้นๆ ออกและนำเอาลายเซ็นของตนเซ็นลงไปในเมื่อจะเก็บข้อความนั้นลงในล็อกไฟล์ ทั้งคอนฟิกูเรชันไฟล์และฐานข้อมูลสามารถเก็บไว้ที่ส่วนกลางบนฝั่งของเครื่องแม่ข่ายได้และสามารถดาวน์โหลดไปใช้โดยเครื่องลูกข่ายในขณะที่ทำการ startup เครื่องได้อีกด้วย

7.3.2.2 การไม่แสดงตัวตนว่ามีการใช้งานอยู่ (stealth)

เซมเฮน มีความสามารถในการอำพรางตัวไม่ให้ผู้บุกรุกสามารถรู้ได้ว่ามีโปรเซสของเซมเฮนกำลังทำงานอยู่ในระบบซึ่งการทำงานในลักษณะนี้เรียกว่าทำงานแบบ stealth ซึ่งสามารถทำได้โดยการ คอมไพล์เซมเฮนให้สนับสนุนโหมด stealth แต่การทำงานในโหมด stealth นั้นก็ยังอาจสามารถถูกตรวจสอบได้จากการค้นหาคอนฟิกูเรชันไฟล์หรือค้นหาจากสตริง ดังนั้นเซมเฮนจึงมีอุปสรรคที่เพิ่มเติมเข้ามาคือ

- สตริงที่สามารถอ่านได้ซึ่งอยู่ใน executable file, ล็อกไฟล์ และในฐานข้อมูลสามารถทำให้ดูไม่รู้เรื่องได้ตัวอย่างเช่น อาจทำให้มองดูแล้วเหมือนเป็นข้อมูลใบนารี
- สามารถยกเลิกการใช้ command line parsing ได้

- สามารถทำข้อมูลคอนฟิกรูเรชั่นต่างๆ ให้เป็นรูปภาพโดยใช้ steganography
- Executable สามารถบีบอัดและเข้ารหัสลับได้
- ฐานข้อมูลและล็อกไฟล์อาจถูกนำไปซ่อนโดยการนำไปต่อท้ายภาพที่มีอยู่แล้วได้โดยภาพนั้นยังคงดูเหมือนปกติ

7.3.2.3 คุณสมบัติในการทำงานของ Samhain

- สามารถ checksum ได้หลากหลายรูปแบบ เช่น TIGER192, SHA-1 ,MD5 เป็นต้น โดยจะตรวจสอบได้จากหลาย ส่วน เช่น size, mode/permission, owner, group, creation/modification/access time, inode, number of hardlinks, linked path(symbolic links) major/minor device number
- สามารถใช้ shell wildcard pattern ในการระบุไฟล์หรือไดเรกทอรีที่จะตรวจสอบได้
- สามารถตั้งและปรับแต่ง policy ของระบบได้ 8 รูปแบบที่แตกต่างกัน
- การตรวจสอบในเชิงลึก สามารถตั้งค่าให้ตรวจสอบทั้งหมดหรือเฉพาะส่วนได้
- สามารถตรวจสอบความเปลี่ยนแปลงของระบบได้ ว่ามีไฟล์ใดถูกแก้ไข หรือถูกลบไป
- สามารถตรวจสอบความถูกต้องของเคอร์เนลที่กำลังใช้งานอยู่ได้ เพื่อตรวจสอบรูทคิตได้
- สามารถตรวจสอบการเปลี่ยนแปลงหรือสร้าง SUID/SGID files โดยสามารถสั่งให้ตรวจสอบอย่างสม่ำเสมอตามเวลาที่กำหนดได้
- สามารถตรวจสอบการ mount ต่างๆ รวมทั้งการ mount files system
- สามารถตรวจสอบการ login/logout ของผู้ใช้งานได้ โดยใช้ system file ที่ชื่อว่า utmp
- สามารถกำหนดให้มีการจัดเก็บและแสดงการเตือนได้
- เครื่องแม่ข่ายฐานข้อมูลจะได้รับข้อมูลผ่านทาง TCP connection ที่มีการเข้ารหัสลับไว้ และมีการพิสูจน์ตัวตนระหว่างกัน เพื่อให้แน่ใจได้ว่าเป็นตัวจริงทั้งคู่
- สามารถใช้งาน Syslog ได้
- สามารถใช้งานผ่านทางคอนโซลได้ถ้ามีการติดตั้งตัวเดมอนไว้
- ล็อกไฟล์ที่จัดเก็บจะมีการ Sign ไว้เพื่อป้องกันการแก้ไขของผู้ไม่มีสิทธิ์
- อีเมลล์ที่ใช้จะมีการ Sign ไว้เช่นเดียวกัน
- รองรับตัว RDBMS ที่หลากหลาย เช่น MySQL, PostgreSQL, และ Oracle

7.3.3 PortSentry

โปรแกรม PortSentry เป็นโปรแกรมสำหรับลินุกซ์และโอเอสตระกูลยูนิกซ์ที่ออกแบบขึ้นเพื่อตรวจสอบการสแกนพอร์ตจากภายนอก และสามารถตอบสนองต่อการกระทำนั้น ๆ ได้หลายรูปแบบ พัฒนาขึ้นโดยเป็นส่วนหนึ่งของ Abacus Project ซึ่งริเริ่มโดย Craig H. Rowland โดย

ลักษณะแล้วถือว่าเป็นซอฟต์แวร์ประเภท Network based Intrusion Detection (NIDS) ซึ่งนอกจากจะมีโหมดการตรวจสอบการสแกนพอร์ตให้เลือกใช้งานถึง 6 โหมดแล้ว ยังสามารถได้ตอบการสแกนพอร์ตนั้นได้หลายแบบอีกด้วย เช่น

- บันทึกข้อมูลของการสแกนพอร์ตที่เกิดขึ้น และรายงานผ่านบริการ syslogd ซึ่งโดยปกติจะสามารถย้อนตรวจดูเหตุการณ์ย้อนหลังได้ที่ไฟล์ /var/log/messages หรืออาจจะเปลี่ยนไปบันทึกที่ไฟล์อื่นแยกออกไปก็ได้ตามต้องการ

```

root@icute:~
File Edit View Terminal Go Help
[root@icute root]# tail /var/log/messages
Oct  9 16:26:19 icute kernel: cdrom: This disc doesn't have any tracks I recognize
Oct  9 16:57:33 icute portsentry[3534]: attackalert: SYN/Normal scan from host: 192.168.0.6/192.168.0.6 to TCP port: 20
Oct  9 16:57:33 icute portsentry[3534]: attackalert: Host 192.168.0.6 has been blocked via wrappers with string: "ALL: 192.168.0.6"
Oct  9 16:57:33 icute portsentry[3534]: attackalert: Host 192.168.0.6 has been blocked via dropped route using command: "n/route add -host 192.168.0.6 reject"
Oct  9 16:57:33 icute portsentry[3534]: attackalert: SYN/Normal scan from host: 192.168.0.6/192.168.0.6 to TCP port: 23
Oct  9 16:57:33 icute portsentry[3534]: attackalert: Host: 192.168.0.6/192.168.0.6 is already blocked Ignoring
Oct  9 16:57:34 icute portsentry[3534]: attackalert: SYN/Normal scan from host: 192.168.0.6/192.168.0.6 to TCP port: 24
Oct  9 16:57:34 icute portsentry[3534]: attackalert: Host: 192.168.0.6/192.168.0.6 is already blocked Ignoring
Oct  9 16:57:34 icute portsentry[3534]: attackalert: SYN/Normal scan from host: 192.168.0.6/192.168.0.6 to TCP port: 21
Oct  9 16:57:34 icute portsentry[3534]: attackalert: Host: 192.168.0.6/192.168.0.6 is already blocked Ignoring
[root@icute root]#
  
```

รูปที่ 7.5 รายงานการถูกสแกนพอร์ตโดย Syslogd

- ปิดกั้นโฮสต์ฝ่ายตรงข้ามโดยอัตโนมัติ หรือ Automatic Real time blocking ด้วย TCP Wrappers โดยจะอัปเดตที่ไฟล์ /etc/hosts.deny ให้เองทันที เช่น เมื่อถูกเครื่อง 203.182.16.1 สแกนเข้ามา ไฟล์ /etc/hosts.deny จะมีข้อความเกิดขึ้นเองดังนี้

ALL: 203.182.16.1

- คอนฟิก kernel route table โดยอัตโนมัติ เพื่อตัดการติดต่อกับโฮสต์ฝ่ายตรงข้าม

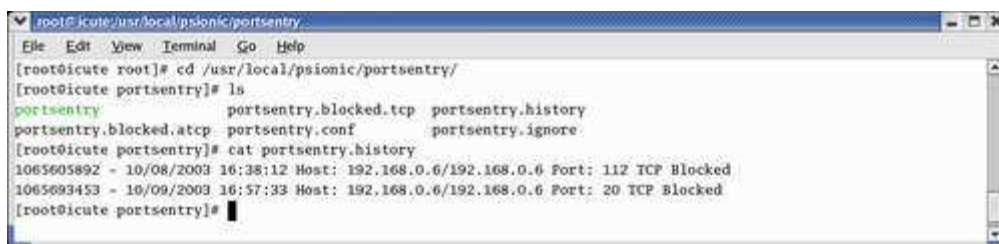
```

root@icute:~
File Edit View Terminal Go Help
[root@icute root]# route -n
Kernel IP routing table
Destination Gateway Genmask Flags Metric Ref Use Iface
192.168.0.6 - 255.255.255.255 !H 0 - 0 -
192.168.0.0 0.0.0.0 255.255.255.0 U 0 0 0 eth0
169.254.0.0 0.0.0.0 255.255.0.0 U 0 0 0 eth0
127.0.0.0 0.0.0.0 255.0.0.0 U 0 0 0 lo
0.0.0.0 192.168.0.200 0.0.0.0 UG 0 0 0 eth0
[root@icute root]#
  
```

รูปที่ 7.6 Kernel Routing Table ภายหลังจากถูกเครื่อง 192.168.0.6 สแกนเข้ามา

- ระบบ History เป็นการบันทึกผลการตัดการติดต่อที่ได้กระทำไปแล้ว โดยมีข้อมูลเกี่ยวกับวันเวลาที่มีการสแกนพอร์ตเกิดขึ้น ชื่อและไอพีแอดเดรสของโฮสต์ฝ่ายตรง

ข้าม (สามารถกำหนดให้ใช้ระบบ DNS Lookup หรือไม่ได้ตามต้องการ) โดยจะเก็บไว้ในไฟล์ `portsentry.history`



```

root@icute:/usr/local/psionic/portsentry
[root@icute root]# cd /usr/local/psionic/portsentry/
[root@icute portsentry]# ls
portsentry          portsentry.blocked.tcp  portsentry.history
portsentry.blocked.atcp  portsentry.conf        portsentry.ignore
[root@icute portsentry]# cat portsentry.history
1065605892 - 10/08/2003 16:38:12 Host: 192.168.0.6/192.168.0.6 Port: 112 TCP Blocked
1065683453 - 10/09/2003 16:57:33 Host: 192.168.0.6/192.168.0.6 Port: 20 TCP Blocked
[root@icute portsentry]#

```

รูปที่ 7.7 ไฟล์ History ของ PortSentry

- Prioritize Response เป็นความสามารถใหม่มีตั้งแต่เวอร์ชัน 1.1 ขึ้นไป คือ สามารถจัดระดับความสำคัญของการจัดการต่อการสแกนพอร์ตได้ และยังสามารถตั้งค่าสั่งภายนอกได้ตามต้องการ จึงมีประโยชน์มากต่อการสร้างระบบแจ้งเตือน (Alert and Alarm) ในรูปแบบที่ผู้ดูแลระบบสามารถกำหนดขึ้นได้เอง

ในการนำ `portsentry` มาใช้ในโครงการนี้ จะต้องทำการแก้ไขค่าคอนฟิกใด ๆ ทั้งโปรแกรมไฟล์คอนฟิก และ ไฟล์ History โดยจะอยู่รวมกันที่ `/usr/local/psionic/portsentry` เมื่อต้องการเรียกใช้งานจะใช้คำสั่งดังนี้

./portsentry -atcp

ซึ่งอาร์กิวเมนต์ `-atcp` จะเป็นการกำหนดให้ PortSentry ทำงานในโหมด Advanced Stealth Detection ซึ่งจะซ่อนการทำงานไว้ได้ แม้แต่ Localhost เองก็ไม่รู้ว่ามีเปิด Socket จาก PortSentry ไว้ (โหมดนี้ทำงานได้เฉพาะ Linux Kernel เท่านั้น หากนำไปรันบนระบบอื่น ๆ จะไม่สามารถทำงานได้)

7.4 การทำงานของโปรแกรม

ไฟร์แตรปจะรอรับการติดต่อจากเซิร์ฟเวอร์ของโปรแกรม S2I ที่รันอยู่บนไฟร์สกรีน ผ่านช่องทาง SSH เพื่อให้ไฟร์สกรีนตรวจสอบสภาพความอ่อนแอของตัวเครื่องกับดักในไฟร์แตรป และยังรอรับเส้นทางที่จะถูกสร้างขึ้นจาก S2I เพื่อให้ผู้บุกรุกเข้ามา เมื่อผู้บุกรุกถูกส่งเข้ายังเครื่องไฟร์แตรปแล้ว โปรแกรมเซเบคไคลเอนต์ที่อยู่ในไฟร์แตรป จะทำการบันทึกพฤติกรรมต่าง ๆ ที่ผู้บุกรุกกระทำต่อไฟร์แตรป เช่น keystroke , ไฟล์ต่าง ๆ ที่ผู้บุกรุกนำเข้ามา หรือนำออกไป

นอกจากนี้เพื่อความแนบเนียนในการซ่อนตัวจากผู้บุกรุกของไฟร์แตรป จึงมีการสร้างสคริปต์ในการปลอมไอพีแอดเดรสของเครื่องกับดัก ให้เป็นไอพีแอดเดรสของเครื่องเว็บเซิร์ฟเวอร์จริง เช่น สคริปต์ลอกเมื่อใช้คำสั่ง `ifconfig` , `traceroute` หรือ `ping` เพื่อให้ผู้บุกรุกไม่สงสัยว่าตนเองอยู่ในเครื่องกับดัก